



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



LAB MANUAL

Functions in Python

Introduction

Functions are one of the most important concepts in Python programming. As programs become larger and more complex, writing all code in a single block becomes difficult to manage, understand, and maintain. Functions solve this problem by allowing programmers to divide a program into smaller, reusable sections of code that perform specific tasks.

A function is a named block of code that executes only when it is called. Once a function is defined, it can be used multiple times throughout a program without rewriting the same code. This promotes code reusability, reduces redundancy, and makes programs more organized.

Functions are widely used in software development, data science, artificial intelligence, machine learning, web development, and automation. Understanding functions is essential for becoming a proficient Python programmer.

Learning Objectives

After completing this lab manual, students will be able to:

- Understand the concept and importance of functions.
- Create and call user-defined functions.
- Work with parameters and arguments.



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



-
- Return values from functions.
 - Differentiate between local and global variables.
 - Implement recursive functions.
 - Create and use lambda functions.
 - Apply functions to solve practical programming problems.
-

Theory Notes

Understanding Functions

A function is a reusable block of code that performs a specific task. Instead of writing the same code repeatedly, programmers can place the code inside a function and execute it whenever required.

Consider a situation where a program needs to display a welcome message multiple times. Without functions, the same code must be written repeatedly. With functions, the code is written once and called whenever needed.

Functions help programmers organize code into logical units. Each function is responsible for performing a single task, making programs easier to understand and maintain.

Why Functions are Important

Functions provide several advantages in programming.

Code reusability allows a function to be written once and used many times. This reduces duplication and makes modifications easier because changes only need to be made in one place.



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Functions improve readability by breaking large programs into smaller sections. When code is divided into meaningful functions, it becomes easier for other programmers to understand the program's purpose.

Functions also simplify testing and debugging. Errors can be isolated within individual functions, making troubleshooting faster and more efficient.

Creating a Function

In Python, functions are created using the `def` keyword. The function definition consists of the function name, parentheses, and a block of code.

Syntax

```
def function_name():  
    # Function body
```

Example

```
def welcome():  
    print("Welcome to Python Programming")
```

The function above is defined but not executed. To run the code inside the function, it must be called.

Calling a Function



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



After defining a function, it can be executed by using its name followed by parentheses.

Example

```
def welcome():  
  
    print("Welcome to Python Programming")  
  
welcome()
```

Output

```
Welcome to Python Programming
```

Every time the function is called, the statements inside it are executed.

Parameters and Arguments

Functions often require input values to perform specific operations. These inputs are called parameters.

Parameters are variables listed in the function definition, while arguments are the actual values passed during the function call.

Example

```
def greet(name):  
  
    print("Hello", name)
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



```
greet("Ali")
```

Output

```
Hello Ali
```

In this example, `name` is the parameter and `"Ali"` is the argument.

Multiple Parameters

A function may accept multiple parameters.

Example

```
def add(a, b):  
    print(a + b)
```

```
add(10, 20)
```

Output

```
30
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



The values are assigned to parameters based on their position.

Return Statement

Many functions perform calculations and return results to the caller. The **return** statement is used for this purpose.

Example

```
def square(number):  
    return number * number  
  
result = square(5)  
print(result)
```

Output

25

The return statement sends the calculated value back to the calling program.

Positional Arguments

In positional arguments, values are assigned according to their order.



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Example

```
def student(name, age):  
    print(name, age)  
  
student("Ahmed", 20)
```

Output

Ahmed 20

The first value is assigned to **name** and the second value is assigned to **age**.

Keyword Arguments

Keyword arguments allow arguments to be passed using parameter names.

Example

```
def student(name, age):  
    print(name, age)  
  
student(age=20, name="Ahmed")
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Output

Ahmed 20

Keyword arguments improve readability and eliminate concerns about argument order.

Default Parameters

Default parameters allow a function to use predefined values if no argument is provided.

Example

```
def greet(name="Guest") :  
    print("Hello", name)
```

```
greet()
```

```
greet("Ali")
```

Output

```
Hello Guest
```

```
Hello Ali
```

Default values make functions more flexible and easier to use.



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Variable-Length Arguments

Sometimes the number of arguments is unknown. Python provides `*args` and `**kwargs` for such situations.

Using `*args`

```
def total(*numbers):  
    print(sum(numbers))
```

```
total(10, 20, 30, 40)
```

Output

```
100
```

The `*args` parameter collects multiple values into a tuple.

Using `**kwargs`

```
def student(**data):  
    print(data)
```

```
student(name="Ali", age=20)
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Output

```
{'name': 'Ali', 'age': 20}
```

The ****kwargs** parameter collects keyword arguments into a dictionary.

Variable Scope

Variable scope determines where a variable can be accessed within a program.

Python primarily supports local and global variables.

Local Variables

A local variable is created inside a function and can only be accessed within that function.

Example

```
def show():  
    message = "Python"  
    print(message)
```

```
show()
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



The variable `message` exists only while the function is executing.

Global Variables

A global variable is defined outside a function and can be accessed from different parts of the program.

Example

```
country = "Pakistan"
```

```
def display():  
    print(country)
```

```
display()
```

Output

```
Pakistan
```

Global variables remain available throughout the program.



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Recursive Functions

A recursive function is a function that calls itself.

Recursion is useful for solving problems that can be broken into smaller versions of the same problem.

Example

```
def countdown(n):  
    if n == 0:  
        return  
  
    print(n)  
    countdown(n - 1)
```

```
countdown(5)
```

Output

```
5  
4  
3  
2  
1
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Every recursive function must have a base condition to prevent infinite recursion.

Lambda Functions

A lambda function is a small anonymous function created using the `lambda` keyword. Lambda functions are generally used when a simple function is needed for a short period of time.

Syntax

```
lambda arguments: expression
```

Unlike regular functions, lambda functions can contain only a single expression.

Example

```
square = lambda x: x * x
```

```
print(square(5))
```

Output

25

Lambda functions are often used with higher-order functions such as `map()`, `filter()`, and `sorted()`.



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Lambda with map()

The `map()` function applies an operation to every item in an iterable.

Example

```
numbers = [1, 2, 3, 4]
```

```
result = list(map(lambda x: x * 2, numbers))
```

```
print(result)
```

Output

```
[2, 4, 6, 8]
```

Lambda with filter()

The `filter()` function selects elements that satisfy a condition.

Example

```
numbers = [1, 2, 3, 4, 5, 6]
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



```
result = list(filter(lambda x: x % 2 == 0, numbers))
```

```
print(result)
```

Output

```
[2, 4, 6]
```

Lambda with sorted()

Lambda functions are frequently used for custom sorting operations.

Example

```
students = [  
    ("Ali", 85),  
    ("Ahmed", 75),  
    ("Sara", 95)  
]  
  
students.sort(key=lambda x: x[1])
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



```
print(students)
```

The list is sorted according to marks.

Syntax Reference Sheet

Basic Function

```
def function_name():  
    pass
```

Function with Parameters

```
def function_name(parameter1, parameter2):  
    pass
```

Function with Return Value

```
def function_name():  
    return value
```




**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Default Parameter

```
def function_name(parameter="Default"):  
    pass
```

Variable-Length Arguments

```
def function_name(*args):  
    pass  
  
def function_name(**kwargs):  
    pass
```

Recursive Function

```
def function_name():  
    function_name()
```

Lambda Function

```
lambda arguments: expression
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Worked Examples

Example 1: Addition Function

```
def add(a, b):  
    return a + b  
  
print(add(10, 20))
```

Output

30

Example 2: Finding Area of Rectangle

```
def area(length, width):  
    return length * width  
  
print(area(5, 4))
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Output

20

Example 3: Recursive Factorial

```
def factorial(n):  
  
    if n == 1:  
        return 1  
  
    return n * factorial(n - 1)  
  
print(factorial(5))
```

Output

120



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Common Errors and Troubleshooting

One common mistake is forgetting parentheses while calling a function. A function definition alone does not execute the code. The function must be called.

Another frequent error is incorrect indentation. Python relies on indentation to identify blocks of code. Improper indentation results in syntax errors.

Students often provide an incorrect number of arguments during function calls. The number of arguments supplied should match the number of required parameters.

Another issue occurs when a function is expected to return a value but the return statement is missing. In such cases, Python returns **None**.

Recursive functions may also cause problems if a base condition is not provided, leading to infinite recursion and program crashes.

Programming Exercises

Beginner Level

1. Create a function that displays your name.
 2. Create a function that prints "Welcome to Python".
 3. Create a function that adds two numbers.
 4. Create a function that calculates the square of a number.
 5. Create a function that determines whether a number is even or odd.
-

Intermediate Level



1. Create a function to calculate the area of a rectangle.
 2. Create a function to find the largest of three numbers.
 3. Create a function to calculate the average of five numbers.
 4. Create a function that checks whether a number is prime.
 5. Create a function that counts the vowels in a string.
-

Advanced Level

1. Create a recursive function to calculate the factorial.
 2. Create a recursive function to calculate Fibonacci numbers.
 3. Use lambda functions with `map()`.
 4. Use lambda functions with `filter()`.
 5. Sort a list of tuples using lambda functions.
-

Lab Activities

Activity 1: Student Grade Calculator

Create a function that accepts marks and returns a grade according to predefined grading criteria.

Expected Steps:

1. Define the function.
 2. Accept marks as a parameter.
 3. Apply grading conditions.
 4. Return the grade.
 5. Display the result.
-



Activity 2: Number Analyzer

Create a function that determines whether a number is positive, negative, or zero.

Activity 3: List Operations Using Lambda Functions

Create a list of numbers and perform the following tasks:

- Double each value using `map()`.
 - Filter even numbers using `filter()`.
 - Sort values using `sorted()`.
-

Assessment

Part A: Theory

1. Define a function.
2. Explain the purpose of the `return` statement.
3. Differentiate between local and global variables.
4. What are lambda functions?
5. Explain recursion with an example.

Part B: Programming

1. Write a function to add two numbers.
2. Write a function to calculate the cube of a number.
3. Write a recursive function to calculate factorial.



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



-
4. Use lambda and `map()` to square all numbers in a list.
 5. Create a function that determines whether a number is prime.
-

Conclusion

Functions are a fundamental component of Python programming. They enable programmers to organize code into reusable and manageable units, making programs easier to develop, understand, and maintain. Mastering functions prepares students for advanced topics such as object-oriented programming, data science, machine learning, artificial intelligence, web development, and software engineering. A strong understanding of functions is essential for building efficient and professional Python applications.